

Dunaújváros Egyetem Bánki Donát Technikum

Projekt feladat dokumentáció

Tartalom

1. A projekt bemutatása és az ötlet leírása.....	2
2. Fejlesztési környezet és alkalmazott technológiák.....	2
3. Felhasználói felület (GUI) megtervezése	3
4. A program működése és a funkciók megvalósítása.....	4
5. Tesztelési folyamat és hibakezelés	7
6. Önreflexió.....	7

Tantárgy neve: Programfejlesztés

Név: Tóth Levente

Osztály: 13.B

Projekt címe: Jegyfoglaló rendszer

1. A projekt bemutatása és az ötlet leírása

A programfejlesztési projektem célja egy egyszerű, asztali környezetben futó, vizuális jegyfoglaló és nyilvántartó alkalmazás (Ticket Reservation System) létrehozása volt. Napjainkban a rendezvényszervezők és adminisztrátorok számára elengedhetetlen egy olyan szoftver, amellyel gyorsan és átláthatóan rögzíthetik a résztvevők adatait és igényeit.

A szoftver alapötlete: A felhasználó (adminisztrátor) egy lenyíló listából kiválaszthatja az aktuális eseményt, majd megadhatja a jegyvásárló nevét, e-mail címét, illetve a lefoglalni kívánt ülőhelyek számát. A program ezt követően a rögzített adatokat egy átlátható listában jeleníti meg.

Főbb funkciók:

- **Adatbevitel:** Különböző típusú beviteli mezők (szöveg, szám, lista) a pontos adatfelvétel érdekében.
- **Foglalás rögzítése (Book):** Az adatok érvényesítése után a foglalás bekerül a rendszerbe.
- **Adatok törlése (Delete):** Lehetőség van a tévesen felvitt foglalások eltávolítására.
- **Űrlap ürítése (Clear):** A beviteli mezők egy gombnyomással alaphelyzetbe állíthatóak a gyorsabb újrafelvétel érdekében.

2. Fejlesztési környezet és alkalmazott technológiák

A szoftver asztali alkalmazásként készült el. A fejlesztés során a legmodernebb eszközöket és keretrendszereket használtam, hogy a program gyors, stabil és könnyen bővíthető legyen.

- **Integrált fejlesztői környezet:** Microsoft Visual Studio 17-es verzió (17.14).
- **Programozási nyelv:** C# (C-Sharp)
- **Célplatform és Keretrendszer:** .NET 8.0 Windows környezetre optimalizálva (net8.0-windows).
- **Felhasználói felület:** Windows Forms (WinForms) technológia.

A program fordítása (build folyamata) sikeresen és rendkívül gyorsan, kevesebb mint egy másodperc alatt lezajlott, ami a .NET 8.0 architektúra optimalizáltságának is köszönhető. A projektfájlok a saját, logikusan felépített fejlesztői könyvtáramban kerültek elmentésre (D:\Banki\Tóth Levente 13.B\Program fejlesztés\Jegyfoglaló rendszer\...).

3. Felhasználói felület (GUI) megtervezése

The screenshot shows a Windows-style application window titled "Ticket Reservation System". Inside the window, the title "Ticket Reservation System" is displayed at the top. Below the title, there are four input fields on the left and three buttons on the right. The input fields are labeled "Event:", "Customer Name:", "Email:", and "Seats:". The "Event:" field is a dropdown menu, "Customer Name:" and "Email:" are text boxes, and "Seats:" is a numeric up-down control showing the value "1". The buttons are labeled "Book", "Delete", and "Clear". Below the input fields is a large, empty gray rectangular area.

A grafikus felület tervezésénél a letisztult, "felhasználóbarát" (user-friendly) kialakítást tartottam szem előtt. Az elemek logikusan, fentről lefelé és balról jobbra haladva követik a munkafolyamatot.

A felületen elhelyezett vezérlők (Controls) és funkciójuk:

1. **Címkék (Labels):** Tájékoztatják a felhasználót, hogy melyik mezőbe milyen adatot várunk (Event, Customer Name, Email, Seats).
2. **ComboBox (Esemény választó):** Egy lenyíló menü, amely tartalmazza a választható rendezvényeket. Ezzel elkerülhető a gépelési hiba.
3. **TextBox (Név és E-mail):** Hagyományos szöveges beviteli mezők az ügyfélazonosításhoz.
4. **NumericUpDown (Ülőhelyek száma):** Ez a vezérlő biztosítja, hogy csak számértéket (minimum 1-et) lehessen megadni, megelőzve az érvénytelen (pl. szöveges) bevitt a székek számánál.
5. **Gombok (Buttons - Book, Delete, Clear):** A műveletek végrehajtásáért felelős interaktív elemek.

6. **Kijelző panel (DataGridView / ListBox):** Az ablak alsó részét elfoglaló tágas terület, ahol a sikeres foglalások táblázatos vagy listás formában megjelennek, valós idejű visszacsatolást adva az adminisztrátornak.

4. A program működése és a funkciók megvalósítása

A program eseményvezérelt (event-driven) logikára épül. Ez azt jelenti, hogy a kód egy-egy része csak akkor fut le, ha a felhasználó valamilyen interakciót végez a felülettel (például rákattint egy gombra).

Az alábbiakban bemutatom a legfontosabb funkciók forráskódját C# nyelven.

A) A foglalás rögzítése (A "Book" gomb kódja) A "Book" gomb megnyomásakor a program először ellenőrzi, hogy a felhasználó minden kötelező adatot megadott-e. Ha a validáció sikeres, a rendszer hozzáadja a beírt adatokat az alsó listához (DataGridView).

C#

```
private void btnBook_Click(object sender, EventArgs e)
{
    // Adatok kinyerése a vezérlőkből

    string selectedEvent = cmbEvent.Text;

    string customerName = txtCustomerName.Text;

    string email = txtEmail.Text;

    int seats = (int)numSeats.Value;

    // Bemeneti adatok ellenőrzése (Validáció)
    if (string.IsNullOrEmpty(selectedEvent) ||
        string.IsNullOrEmpty(customerName) ||
        string.IsNullOrEmpty(email))
    {
        MessageBox.Show("Kérjük, töltsön ki minden mezőt a foglaláshoz!", "Figyelmeztetés",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    // Új sor hozzáadása a táblázathoz
```

```
dgvReservations.Rows.Add(selectedEvent, customerName, email, seats);
```

```
// Visszajelzés a felhasználónak
```

```
MessageBox.Show("A foglalás sikeresen rögzítve!", "Siker", MessageBoxButtons.OK,  
MessageBoxIcon.Information);
```

```
// Mezők automatikus ürítése sikeres foglalás után
```

```
ClearFields();
```

```
}
```

B) A beviteli mezők ürítése (A "Clear" gomb kódja) A téves bevitel vagy új foglalás megkezdése esetén hasznos a mezők alaphelyzetbe állítása. Ezt egy külön metódusba szerveztem, hogy a kódot máshonnan is meg lehessen hívni (mint a fenti foglalás végén).

C#

```
private void btnClear_Click(object sender, EventArgs e)
```

```
{
```

```
    ClearFields();
```

```
}
```

```
private void ClearFields()
```

```
{
```

```
    // A vezérlők értékének alaphelyzetbe állítása
```

```
    cmbEvent.SelectedIndex = -1;
```

```
    txtCustomerName.Clear();
```

```
    txtEmail.Clear();
```

```
    numSeats.Value = 1; // Az alapértelmezett helyek száma 1
```

```
    // A fókusz visszahelyezése az első mezőre
```

```
    cmbEvent.Focus();
```

```
}
```

C) Foglалás törlése (A "Delete" gomb kódja) Amennyiben egy foglalást lemondanak, vagy hibásan lett felvive, a listában (DataGrid) való kijelölés után törölhető.

C#

```
private void btnDelete_Click(object sender, EventArgs e)
{
    // Ellenőrizzük, hogy van-e kijelölt sor a táblázatban
    if (dgvReservations.SelectedRows.Count > 0)
    {
        // Megerősítés kérése törlés előtt
        var confirmResult = MessageBox.Show("Biztosan törölni szeretné a kijelölt foglalást?",
            "Törlés megerősítése",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

        if (confirmResult == DialogResult.Yes)
        {
            // A kijelölt sor eltávolítása
            foreach (DataGridViewRow item in this.dgvReservations.SelectedRows)
            {
                dgvReservations.Rows.RemoveAt(item.Index);
            }
        }
    }
    else
    {
        MessageBox.Show("Kérjük, válasszon ki egy törlendő foglalást a listából!", "Hiba",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

5. Tesztelési folyamat és hibakezelés

A fejlesztés utolsó fázisában alapos tesztelésnek vettem alá az alkalmazást. A szoftver tesztelése során az úgynevezett "Feketedoboz-tesztelés" (Black-box testing) módszerét alkalmaztam, ahol a felhasználói funkciókat próbáltam végig.

A tesztelt forgatókönyvek:

1. **Üresen hagyott mezők kezelése:** Ha a "Book" gombra kattintok adatok nélkül, a program nem omlik össze, hanem egy hibaüzenet (MessageBox) formájában figyelmeztet a hiányosságra. (Sikeres teszt).
2. **Extrém számérték bevitele a székekhez:** A NumericUpDown vezérlő tulajdonságainak (Minimum = 1, Maximum = 100) beállításával garantáltam, hogy ne lehessen 0, negatív, vagy irreálisan nagy számú ülőhelyet foglalni.
3. **Adatok törlése kijelölés nélkül:** Ha a "Delete" gombot úgy nyomják meg, hogy nincs elem a listában, a szoftver elegánsan tájékoztat, leállítás (Exception) helyett.
4. **Fordítási hibák:** Az output log alapján a build folyamat 0 hibával és 0 figyelmeztetéssel futott le.

6. Önreflexió

Értékelés és tapasztalatok: A projekt elkészítése során rengeteget fejlődött a C# és a .NET környezettel kapcsolatos tudásom. Kihívást jelentett az objektumok tulajdonságainak (Properties) precíz beállítása és az eseménykezelők (Event Handlers) összekötése, de a végeredmény egy stabil és jól működő program lett. Különösen büszke vagyok a szoftver letisztult megjelenésére és a logikus hibakezelésre.

Összességében a Jegyfoglaló rendszer projekt egy kiváló gyakorlati feladat volt, amely megmutatta, hogyan lehet elméleti programozási tudást átültetni egy a való életben is használható, kézzelfogható szoftverre.